

L04 — Visual Timeline of Computer Graphics Rendering Evolution

Exploring AI-Generated Imagery to Document the History of Rendering Technology

Student	Team	Course	Due Date
Joseph Clay	TuringCollective	ITAI-1370	Jun 27, 2026

■ LIVE INTERACTIVE SUBMISSION

<https://cg-timeline-evolution.netlify.app/>

Click the link above to open the fully deployed visual timeline on Netlify. This PDF serves as the written report companion to the live interactive site.

Introduction

This report documents the evolution of computer graphics rendering technology through a visual timeline of eight major milestones — from the mathematical foundations of ray tracing in the 1960s to the real-time AI-accelerated rendering pipelines of today. For each milestone, a generative AI tool (DALL-E) was used to produce a visual representation of the technology and its visual impact. The interactive version of this timeline is deployed and accessible at the Netlify link above.

Visual Timeline — 8 Milestones in Rendering History

1960s Mathematical Foundations of Ray Casting

In the 1960s, Arthur Appel at IBM developed ray casting — the conceptual precursor to modern ray tracing. Rather than simulating full light interaction, ray casting determined visibility by shooting a single ray per pixel from the camera into the scene. While limited, this established the foundational logic that every modern raytracer still uses: trace a ray from the viewer's eye and calculate what it hits.

AI Prompt used: "A 1960s mainframe computer with a wireframe 3D object displayed on a green phosphor monitor, illustrating early ray casting in computer graphics, technical and historical aesthetic"

1980

Turner Whitted Introduces Recursive Ray Tracing

In 1980, Turner Whitted published "An Improved Illumination Model for Shaded Display," introducing recursive ray tracing. This breakthrough allowed rays to bounce off reflective surfaces and refract through transparent materials, producing the first photorealistic computer-generated images with accurate reflections and shadows. It transformed computer graphics from flat approximations into physically grounded light simulation.

AI Prompt used: "A photorealistic glass sphere reflecting a scene of colored spheres on a checkered floor, rendered with recursive ray tracing, 1980s computer graphics style"

1989

Pixar's RenderMan — Offline Production Rendering

Pixar released RenderMan in 1989, a production-grade renderer that became the industry standard for film CGI. RenderMan introduced the concept of procedural shading — programmable surface descriptions that allowed artists to define how materials respond to light. Used in every Pixar film and dozens of major Hollywood productions, RenderMan demonstrated that offline raytracing and shading could produce images indistinguishable from photography.

AI Prompt used: "A 1989 computer graphics rendering workstation with photorealistic CGI output on screen, showing early Pixar-style animation rendering, cinematic quality"

1996

Quake and Real-Time 3D Rasterization

id Software's Quake (1996) popularized real-time 3D rendering using rasterization on consumer hardware. While not ray tracing, this milestone marked the moment 3D rendering moved from workstation laboratories to home computers. The GPU became the driving force of rendering advancement, setting the stage for all future real-time graphics including eventual real-time ray tracing.

AI Prompt used: "A 1996-era first-person shooter video game rendered in real-time 3D, dark dungeon corridors with dynamic lighting and textures, early GPU rasterization style"

2001

Programmable Shaders — DirectX 8 / OpenGL

The introduction of programmable vertex and pixel shaders in DirectX 8 (2001) gave developers direct control over how the GPU processed each vertex and pixel on screen. Previously, rendering pipelines were fixed — artists could not customize how light interacted with surfaces. Programmable shaders unlocked per-pixel lighting, bump mapping, and effects that became the visual standard for the next decade of game development.

AI Prompt used: "A 2001 video game environment with per-pixel lighting, bump-mapped textures and dynamic shadows, programmable shader visual effects, DirectX 8 era aesthetics"

2006

NVIDIA CUDA — GPU General Purpose Computing

NVIDIA's launch of CUDA in 2006 enabled general-purpose computing on the GPU, unlocking the GPU's parallel processing power for tasks beyond graphics. This was a pivotal enabling technology for deep learning and AI — but it also dramatically accelerated offline ray tracing by allowing rendering workloads to be distributed across thousands of GPU cores simultaneously, collapsing render times from hours to minutes.

AI Prompt used: "A modern NVIDIA GPU circuit board with streams of light rays being calculated in parallel, visualizing GPU-accelerated ray tracing computation, technical and futuristic"

2018

NVIDIA RTX — Real-Time Ray Tracing Arrives

In 2018, NVIDIA launched its RTX GPU architecture alongside Microsoft's DirectX Raytracing (DXR) framework, bringing real-time ray tracing to consumer hardware for the first time. Dedicated RT cores in the GPU could calculate ray-surface intersections fast enough to run during gameplay. Games like Battlefield V and Control demonstrated accurate reflections, global illumination, and soft shadows that were previously only possible in offline film rendering.

AI Prompt used: "A cinematic video game scene with photorealistic reflections on wet city streets at night, neon lights reflected in puddles, NVIDIA RTX real-time ray tracing, hyperrealistic"

2020+

AI-Accelerated Rendering — DLSS and Path Tracing

The current frontier of rendering combines ray tracing with AI upscaling. NVIDIA's DLSS (Deep Learning Super Sampling) uses a neural network trained on high-resolution frames to upscale lower-resolution rendered output in real time, allowing full path tracing — the most accurate form of light simulation — at playable frame rates. Cyberpunk 2077's "Overdrive Mode" demonstrated that AI-assisted rendering could produce imagery that rivals pre-rendered CGI while running interactively on consumer hardware.

AI Prompt used: "A futuristic cyberpunk city at night with full path-traced lighting, neon signs, rain, photorealistic reflections and global illumination, AI-enhanced rendering, ultra-realistic"

Reflective Analysis — Generative AI as a Historical Visualization Tool

Using generative AI (DALL-E) to produce visuals for each milestone in rendering history presented both significant advantages and notable limitations worth examining.

Strengths

- **Accessibility:** AI image generation made it possible to produce representative visuals for milestones where actual archival images are difficult to source or under copyright. For early milestones like 1960s ray casting, no photographic record exists — generative AI filled that gap with plausible, evocative imagery.
- **Speed and iteration:** Prompts could be refined and regenerated in seconds, allowing selection of the most representative image from multiple candidates. This would be impossible

with traditional illustration or photography.

- **Visual coherence:** Because all images were generated by the same AI system with consistent prompt structuring, the timeline maintains a visual consistency that helps the viewer focus on the content rather than style differences between sources.

Limitations

- **Technical accuracy:** Generative AI produces aesthetically plausible images, not technically accurate diagrams. An image of "ray tracing" may look compelling without accurately illustrating the mathematical process of ray-surface intersection. For educational purposes, this distinction matters.
- **Anachronism risk:** AI models trained on modern imagery may produce visuals that look contemporary rather than period-accurate. A prompt for "1960s mainframe computer graphics" may still render with a modern visual aesthetic if the model lacks sufficient period-specific training data.
- **Prompt dependency:** The quality of each image is entirely dependent on the specificity of the prompt. Vague prompts produce generic results. Crafting prompts that meaningfully capture a technical milestone requires prior knowledge of the subject — which somewhat undermines the idea of AI as an independent educational resource.

Overall, generative AI proved to be a valuable creative tool for building this visual timeline, particularly for making abstract technical concepts visually tangible for a non-specialist audience. However, it works best as a supplement to — rather than a replacement for — factual research and descriptive annotation. The combination of accurate historical context with AI-generated visual metaphors produced a more engaging and accessible narrative than either approach could achieve alone.

References

- Appel, A. (1968). Some techniques for shading machine renderings of solids. *AFIPS Spring Joint Computer Conference*, 32, 37–45.
- Devarakonda, A. (2026). *ITAI-1370 Module 04 — Lab L04: Explore Generative AI for Visualizing Rendering History*. Houston Community College.
- NVIDIA. (2024). *Ray tracing*. NVIDIA Developer. <https://developer.nvidia.com/discover/ray-tracing>
- OpenAI. (2024). *DALL-E 3*. [AI image generation tool]. <https://openai.com/dall-e-3>
- Pixar Animation Studios. (2024). *RenderMan*. <https://renderman.pixar.com/>
- Unreal Engine. (2024). *What is real-time ray tracing?* <https://www.unrealengine.com/explainers/ray-tracing/what-is-real-time-ray-tracing>
- Whitted, T. (1980). An improved illumination model for shaded display. *Communications of the ACM*, 23(6), 343–349.

Clay, J. (2026). *CG Rendering Evolution — Interactive Visual Timeline* [Web application]. Netlify.
<https://cg-timeline-evolution.netlify.app/>

L04_TuringCollective_JosephClay_ITAI1370 · ITAI-1370 · Prof. Anna Devarakonda · HCC · Summer 2026